

Source Code Structure

How libterae is laid out in src/, include/ and platform/

Emilio Schininà, M.Sc. in Automation Engineering
Version 0.2
November 8, 2026

Copyright © 2026 Emilio Schininà, M.Sc. in Automation Engineering. libterae is licensed under the GNU General Public License, version 3 or any later version, or under a commercial license.

Contents

1 The Three Folders **1**
1.1 What each folder holds. 1
1.2 The layers. 2
1.3 What shapes the folders 4

2 The Headers **7**
2.1 Names 7
2.2 Four kinds of header 8
2.3 The definitions. 8
2.4 What a program includes 10

3 The Protocol Code **11**
3.1 What holds for every file. 11
3.2 The files by layer 11
3.3 The Server and the Client over several files 14

4 The Memory Functions **15**
4.1 The folders 15
4.2 What a folder holds 16
4.3 How `PORT=auto` chooses 17

5 The Platform **19**
5.1 Two levels. 19
5.2 The files at the top 20
5.3 The folders of the systems 22
5.4 The Makefile picks the files. 23

6 Where to Look **25**

1

The Three Folders

The library is three folders of C: `include/`, `src/` and `platform/`. This document describes how they are organised: what each folder is for, how its files are named, which layer of the library each file belongs to, and where to look for a given part of it. It stays at the level of folders and files. What a file does in detail is in its own opening comment, and how to build the library is in *Building libterae*.

The folders are described as they are at version 1.0.5.

1.1 What each folder holds

Table 1.1 The three folders

Folder	What it holds	Built into
`include/`	86 headers: the interface a program reads, the types and constants of the three versions, the format of stored Session state, and what the library needs from a system.	Nothing: a program and the other two folders read them.
`src/`	36 source files: the protocol, the Server Interfaces and the Client Interfaces, with what keeps their Session state across a restart, and the library's version, with no system call, no allocator and no C library.	`libterae.a`, and the shared library of `make shared`
`src/port/`	`memcpy`, `memmove`, `memset` and `memcmp`, for a build with no C library: one folder for each processor, and one in C for any.	`libterae_port.a`, when `PORT` names a folder

continues

source-structure-en.tex Page 2 November 8, 2026 13:46

Table 1.1 The three folders (*continued*)

Folder	What it holds	Built into
`platform/`	52 files: sockets, waiting for them, the clock, random bytes, TLS, and stores in files or in flash. What is the same on every system is at the top of the folder, and what one system does its own way is in a folder of its own.	`libterae.a`, unless `OS=none`, and the shared library

Beside the three folders, a clone has the `Makefile`, which picks the files of a build, `README.md` and `README.zh-CN.md`, with their images in `assets/`, the licences, and `docs/`, which holds the documentation as PDFs, one folder for each language.

1.2 The layers

Figure 1.1 puts the parts of the library in layers. A part calls only the parts of its own layer and of the layers below it, as the arrow on the left says. The brackets on the right say which folder each layer is in. Dashed lines are for what is outside the library.

- **Your program.** Outside the library. It lets `platform/` run the network loop, with `mqtt_net_server_run` or `mqtt_net_client_step`, or it builds with `OS=none` and calls the Server and Client Interfaces itself, from a loop of its own.
- **`platform/`.** The specialisation of the library on a system. It runs the network loop of a Server or a Client: it waits for sockets, reads and writes them, keeps the time, does TLS, keeps the store of Session state in files or in flash, and hands the bytes to the Server and Client Interfaces. The files at the top of the folder are the same on every system. A folder for each system, the `platform/`*`os/`* of the figure, does what that system does its own way, and is the only code of the library that calls a system (Chapter 5).
- **Server Interfaces and Client Interfaces.** The API of the library: what a program, or `platform/`, calls to make a Server or a Client. They take bytes and hand back bytes, and hold every decision the protocol leaves to the Server or the Client. Their words in the figure are the names of their files, as `base_server_route.c` (Chapter 3).
- **Topic tree and Tiers.** Two structures for the layers above: the tree holds the subscriptions and the retained messages of the Server, and the tiers sort the sockets of `platform/` by how busy they are, so that each wait looks at few of them.

source-structure-en.tex Page 3 November 8, 2026 13:46

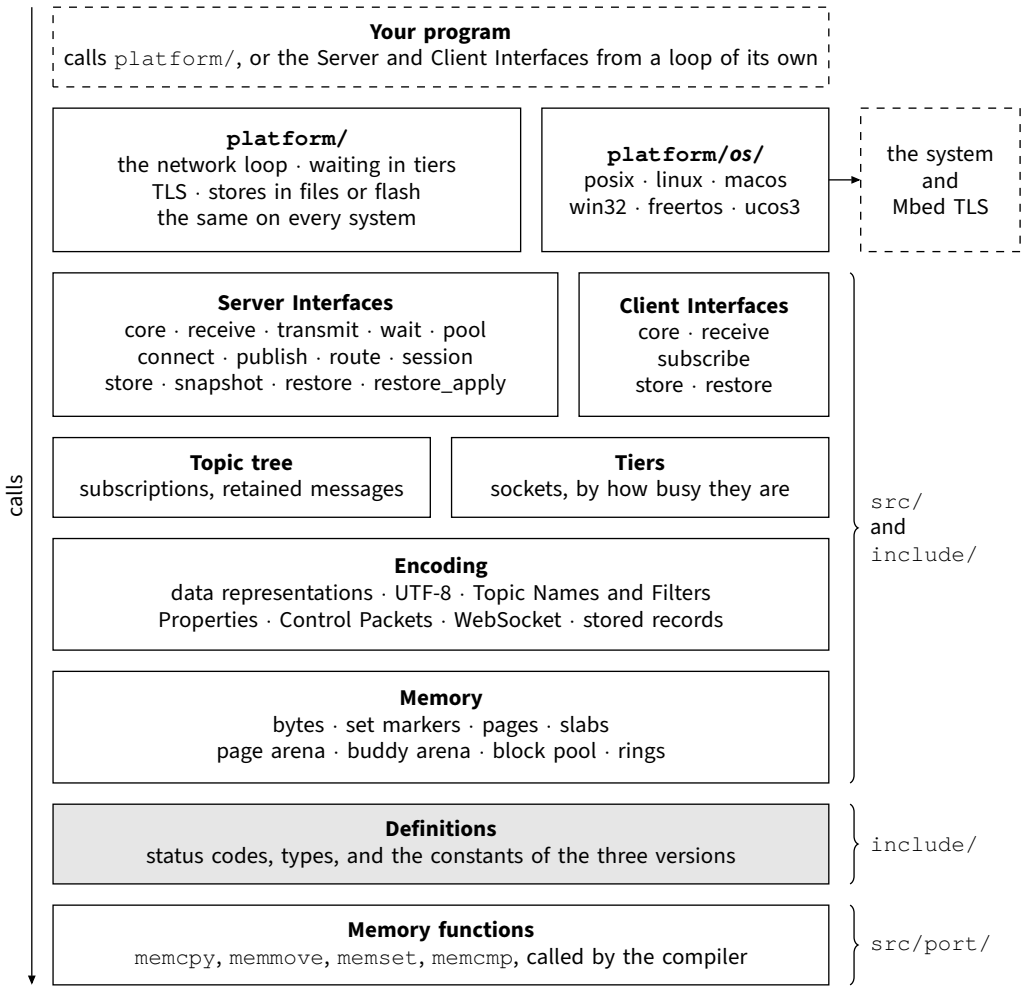


Figure 1.1 The library in layers, with the folder of each

source-structure-en.tex Page 4 November 8, 2026 13:46

- **Encoding.** The data representations of the specifications, the checks of UTF-8 and of topics, the Properties of MQTT 5.0, the decoding and encoding of the Control Packets, the WebSocket frames, and the records of a store, with their checks.
- **Memory.** Copies and comparisons without the C library, markers that say whether memory was set, and the structures that share out the memory a program gives: pages, slabs, two arenas, a pool of blocks and the rings of bytes in transit.
- **Definitions.** Headers of types and constants only, with no source file, used by every layer above them (Chapter 2).
- **Memory functions.** The four functions a compiler may call even in a build with no C library, from `src/port/` (Chapter 4). With a C library, the C library gives them instead.

1.3 What shapes the folders

A few rules hold for all three folders, and explain most of their layout:

- **Flat.** `src/` and `include/` have no subfolders, apart from the folders of `src/port/`. `platform/` has one folder for each system and nothing deeper.
- **Named by version.** A name that starts with `base_` is for what the three versions share. One that starts with `v31_`, `v311_` or `v5_` is for MQTT 3.1, 3.1.1 or 5.0 alone. A file of `platform/` starts with `platform_`, or with the name of its folder, as `win32_socket.c`.
- **One module, one header.** A module is a source file of `src/` and the header of `include/` that declares what it offers, of the same name: `base_pool.c` and `base_pool.h`. The Server and the Client are each spread over several source files, under one header each (Section 3.3).
- **Nothing from a system in `src/`.** `src/` calls nothing outside itself. Whatever differs from one system to another is in `platform/`, behind the headers `base_platform.h`, `base_net.h` and `base_tls.h`.
- **No `#if` names a system.** The top of `platform/` includes no header of a system, and each system's folder uses only what that system documents. A build takes the files of one system because the `Makefile` lists them for its `OS`, not because a condition in the code chooses them.
- **Code from elsewhere only in `src/port/`.** Every file of `src/`, `include/` and `platform/` is libterae's own, apart from the assembly of `src/port/`, where each folder's `NOTICE` says where its files came from and under what licence.

source-structure-en.tex Page 5 November 8, 2026 13:46

- **Every file says what it is.** A file starts with the licence notice, then a comment that says what the file is for and, where it follows the specifications, cites their sections and pages.

2

The Headers

The 86 headers of `include/` are what a program reads, and what `src/` and `platform/` share. They are in one folder, so a program needs only `-Iinclude`. They include nothing of the C library but `<stdint.h>`, `<stdbool.h>` and `<stddef.h>`.

2.1 Names

A header's name says which versions it serves and, through its prefix, how its identifiers start:

Table 2.1 The prefixes of the headers

Prefix	Identifiers	What it is for
`base_`, 42 headers	`MQTT_` and `mqtt_`	What the three versions share.
`v31_`, 9 headers	`MQTT31_`	MQTT 3.1 alone.
`v311_`, 14 headers	`MQTT311_`	MQTT 3.1.1 alone.
`v5_`, 21 headers	`MQTT5_` and `mqtt5_`	MQTT 5.0 alone.

Three endings of a name mark a kind of header:

- **`_t.h`.** One type, and nothing else, so that headers can share it without including more: `base_binary_t.h`, `base_payload_t.h`, `base_run_t.h`, `base_utf8_t.h` and `v5_pair_t.h`.
- **`_check.h`.** The functions that check something the header without the ending defines: `base_utf8_check.h` checks the UTF-8 Encoded Strings whose limits are in `base_utf8.h`, and `base_topic_check.h` the topics whose separator and wildcards are in `base_topic.h`.

source-structure-en.tex Page 8 November 8, 2026 13:46

- **`_internal.h`.** What the source files of one module share, which is not part of the library's interface. `base_store_internal.h` is shared wider: by the stores of the Server and of the Client, and `base_store.c`.

2.2 Four kinds of header

Table 2.2 The kinds of header

Kind	Headers	Count
The interface of a module of `src/`	`base_string.h`, `base_bool.h`, `base_page.h`, `base_slab.h`, `base_page_arena.h`, `base_arena.h`, `base_pool.h`, `base_ring.h`, `base_wire.h`, `base_utf8_check.h`, `base_topic_check.h`, `v5_properties.h`, `base_packet.h`, `base_websocket.h`, `base_tree.h`, `base_tier.h`, `base_store.h`, `base_server.h`, `base_client.h`, `base_library.h`	20
Shared by the source files of a module, not part of the interface	`base_server_internal.h`, `base_client_internal.h`, `base_store_internal.h`	3
Carried out by `platform/`	`base_platform.h`, what the library needs from a system; `base_net.h`, running a Server or a Client on real connections; `base_tls.h`, TLS	3
Definitions: types and constants, with no source file	The headers of Table 2.3	60

2.3 The definitions

Most headers define the types and constants of the specifications, and nothing else. What the versions share and what one of them adds have the same name after the prefix: `base_type.h` has the Control Packet types of all three versions, and `v5_type.h` the one that only MQTT 5.0 has. Table 2.3 lists the names after the prefix, and marks the prefixes each one has a header with. The `type` row, with four marks, stands for `base_type.h`, `v31_type.h`, `v311_type.h` and `v5_type.h`.

source-structure-en.tex Page 9 November 8, 2026 13:46

Table 2.3 The headers of definitions

Name	What it defines	base_	v31_	v311_	v5_
The library					
status	The outcome of a library call	•			
version	The protocol versions	•			
run_t	A run of bytes next to each other in memory	•			
Data representations					
binary_t	Binary Data	•			
utf8_t	UTF-8 Encoded String	•			
utf8	Its limits; the code points it may hold	•		•	•
pair_t	UTF-8 String Pair				•
vbi	Variable Byte Integer				•
The Fixed Header					
fixed	Its layout	•			
type	Control Packet types	•	•	•	•
flags	The flags of PUBLISH; those of the other packets	•	•	•	•
remaining	Remaining Length; that of packets of one size	•	•	•	•
Connecting					
protocol	Protocol Name and level		•	•	•
connect	The Connect Flags; what CONNECT has in one version	•	•	•	•
keep_alive	Keep Alive	•			
client_id	Client Identifier: its length; its characters	•		•	•
connack	Connect Acknowledge Flags			•	•
rc	Return codes, and the Reason Codes of MQTT 5.0		•	•	•
session	Session Expiry Interval				•
port	TCP ports			•	•
websocket	WebSocket subprotocol			•	•

continues

source-structure-en.tex Page 10 November 8, 2026 13:46

Table 2.3 The headers of definitions (*continued*)

Name	What it defines	base_	v31_	v311_	v5_
Publishing and subscribing					
qos	Quality of Service levels	•			
packet_id	Packet Identifier	•			
payload_t	PUBLISH Payload	•			
payload	Payload Format Indicator				•
topic	The separator and wildcards; the length of a name in MQTT 3.1; topics that start with $	•	•	•	•
share	Shared Subscription prefix				•
subscribe	Requested QoS; the Subscription Options of MQTT 5.0		•	•	•
suback	SUBACK return codes		•	•	
Properties					
prop	Property Identifiers				•
prop_value	Property values when absent, and their ranges				•

2.4 What a program includes

- **base_server.h or base_client.h.** The Server Interfaces or the Client Interfaces. Each includes the headers its own types need.
- **base_net.h.** To let platform/ run the loop, and base_tls.h for TLS.
- **base_platform.h.** To use the system's calls that platform/ has for its own use, as mqtt_platform_random_bytes, and to open a store, with mqtt_platform_store_open.
- **base_store.h.** For what a store is made of: the durabilities, the records, and what a restore reports when a store is damaged. base_server.h and base_client.h include it.
- **A version's headers.** For the constants a program compares with, as the Reason Codes of v5_rc.h.

A program is compiled with the page size the library was built with, MQTT_PAGE_SIZE, as *Building libterae* explains: the headers use it for the size of arrays the program gives the library.

3

The Protocol Code

The 36 source files of `src/` are the protocol: the packets, the topics, the memory they live in, the Server and the Client, the store that keeps their Session state across a restart, and the library's version. They are one folder, with `src/port/` beside them (Chapter 4).

3.1 What holds for every file

- **C99, and nothing outside.** Every file is compiled as C99, freestanding, with the compiler's warnings as errors. It makes no system call, calls no allocator and uses no C library: copies and comparisons are `base_string.c`'s own.
- **Memory from the program.** The program gives every array the library uses, at the start, in `mqtt_server_storage` or `mqtt_client_storage` or as arguments. Running out of anything is an ordinary outcome, reported as the protocol allows.
- **No state of its own.** No file has a variable of static storage that it writes. All state lives in structures the program owns and passes in.
- **One thread, and the time as an argument.** Nothing locks, blocks or waits, and a call that needs the time is given it, in milliseconds.
- **MISRA C:2012.** `src/` and `include/` are written to it, and a MISRA checker finds nothing in them. That isn't a claim of compliance.

3.2 The files by layer

Table 3.1 lists the files in the layers of Figure 1.1, from the bottom up. Each file's header has the same name, with `.h` for `.c`, except the files of the Server and of the Client, which their group row names.

source-structure-en.tex Page 12 November 8, 2026 13:46

Table 3.1 The files of `src/`

File	What it does
The library	
`base_library.c`	The version of the library, and the page size it was built with, for a program to check the shared library it runs with.
Memory	
`base_string.c`	Copies, compares, wipes and measures bytes, with no C library.
`base_bool.c`	Markers that tell memory that was set from memory that only holds what was there before, without clearing it.
`base_page.c`	Pages, the unit the Server's memory is cut into, and runs of whole pages.
`base_slab.c`	Slots of one size, grouped by page.
`base_page_arena.c`	Blocks of any size, from pages: the Server's data of varying size.
`base_arena.c`	A buddy allocator over bytes the program gives: the Client's data of varying size.
`base_pool.c`	A pool of blocks of one size, shared by the rings of every connection.
`base_ring.c`	A chain of blocks of the pool, holding the bytes that wait on one connection.
Encoding	
`base_wire.c`	Reads and writes the data representations: integers, Variable Byte Integers, strings and Binary Data.
`base_utf8_check.c`	Checks the characters of a UTF-8 Encoded String.
`base_topic_check.c`	Checks and matches Topic Names and Topic Filters.
`v5_properties.c`	Decodes and encodes the Properties of MQTT 5.0.
`base_packet.c`	Decodes and encodes the Control Packets of the three versions.
`base_websocket.c`	Carries MQTT over WebSocket: the opening handshake and the frames.
`base_store.c`	The format of stored Session state: the headers, the records and their checks, and reading a place of a store.
Topic tree and Tiers	

continues

source-structure-en.tex Page 13 November 8, 2026 13:46

Table 3.1 The files of `src/` (*continued*)

File	What it does
`base_tree.c`	The tree of topic levels that holds the subscriptions and the retained messages.
`base_tier.c`	Sorts sockets by how busy they are, so that each wait looks at few.
Server Interfaces: `base_server.h`, with `base_server_internal.h` between the files	
`base_server.c`	The core: setting up, reading packets and deciding what they mean, the timers.
`base_server_receive.c`	Takes in the bytes each connection sends, and cuts them into packets.
`base_server_transmit.c`	What a connection sends next, and what has gone.
`base_server_wait.c`	How busy each connection is, for the network loop to decide how to wait for it.
`base_server_pool.c`	How many blocks of the pool each connection may take, and waking the connections that wait for them.
`base_server_connect.c`	CONNECT and AUTH, and accepting a connection.
`base_server_publish.c`	PUBLISH from Clients, and the packets that acknowledge it.
`base_server_route.c`	Keeps messages in memory, finds where they go, and turns them into bytes.
`base_server_session.c`	Sessions, subscriptions and retained messages.
`base_server_store.c`	The journal of what outlasts a restart, and the answers that wait for it.
`base_server_snapshot.c`	The snapshot of what outlasts a restart, with which the journal starts again.
`base_server_restore.c`	Putting back what a store holds: reading and checking every place, and choosing what to apply.
`base_server_restore_apply.c`	What each record of a store does to a server being restored.
Client Interfaces: `base_client.h`, with `base_client_internal.h` between the files	
`base_client.c`	What the program calls, the messages held for the Server, and the bytes waiting to go out.
`base_client_receive.c`	Takes the bytes that arrive, and handles the packets the Server sends.

continues

source-structure-en.tex Page 14 November 8, 2026 13:46

Table 3.1 The files of `src/` (*continued*)

File	What it does
`base_client_subscribe.c`	SUBSCRIBE and UNSUBSCRIBE: checks the Topic Filters, takes a Packet Identifier, and holds the packet until its answer comes.
`base_client_store.c`	The journal and the snapshot of its Session State, and the answers that wait for them.
`base_client_restore.c`	Putting back what a store holds.

3.3 The Server and the Client over several files

The Server Interfaces are one header, `base_server.h`, carried out by thirteen source files. What those files share, the functions they call in one another and the layout of the Server's structure, is in `base_server_internal.h`, which a program never includes. The Client Interfaces are `base_client.h` in the same way, carried out by five files that share `base_client_internal.h`. The stores of both, and `base_store.c`, also share `base_store_internal.h`: how a record is put together, and how a restore chooses the snapshot and journal it applies.

A call of the interface starts in one file and may go on through others. `mqtt_server_receive_commit`, in `base_server_receive.c`, cuts the bytes a connection sent into packets; `base_server.c` decides what each one is; a CONNECT goes on to `base_server_connect.c`, a PUBLISH to `base_server_publish.c`, and the message to `base_server_route.c`, which finds its subscribers in the topic tree.

4

The Memory Functions

A compiler may call `memcpy`, `memmove`, `memset` and `memcmp` even in a build with no C library, to copy a structure or to set up an array. `src/port/` has them, for a program that has no C library to give them. With a C library, none of it is built.

4.1 The folders

One folder for each processor, in assembly taken from other projects, and `generic/`, in C, for any processor. `make PORT=`*folder* builds one into `libterae_port.a`, which a program links right after `libterae.a`, and `PORT=auto` lets the compiler choose (Section 4.3).

Table 4.1 The folders of `src/port/`

Folder	Processors	Taken from
`aarch64`	AArch64	Arm Optimized Routines
`arm`	32-bit Cortex-A, little-endian	Arm Optimized Routines, bionic
`armv4t`	ARM7TDMI, ARM9, ARM11, Cortex-R, big-endian Cortex-A	musl, newlib
`armv7m`	Cortex-M3, M4, M7, M33	Apache NuttX, musl
`avr`	AVR	avr-libc
`i386`	x86	musl, newlib
`m68k`	68000 and ColdFire	picolibc
`mips`	MIPS32 with o32	newlib
`powerpc64`	64-bit PowerPC with ELFv2	FreeBSD, NetBSD

continues

source-structure-en.tex Page 16 November 8, 2026 13:46

Table 4.1 The folders of `src/port/` (*continued*)

Folder	Processors	Taken from
`riscv32`	32-bit RISC-V	newlib
`riscv64`	64-bit RISC-V with M	FreeBSD, newlib
`rx`	RX	picolibc
`sh`	SuperH from SH-2	NetBSD
`x86_64`	x86-64	FreeBSD
`xtensa`	Xtensa	newlib
`generic`	Any other, as Cortex-M0, MSP430 or 8051	libterae's own, in C

`sh`, `rx` and `m68k` have been assembled, and `xtensa` built, but none of the four has run on its processor.

4.2 What a folder holds

- **`NOTICE`.** Where each file of the folder came from, at which version, and under what licence, and which files are libterae's own.
- **The files from the other project.** With their original names, as `memcpy.S`, copied byte for byte and never edited. Where they include a header of their project, the folder has a stand-in of the same name, written for libterae, or keeps the path they include, as `machine/asm.h` or `sys/asm.h`.
- **`port_*.S` and `port_*.c`.** libterae's own, and the only files built. Each renames the symbols of a file from the other project and includes it, or, for a function the folder has no assembly for, includes the C version from `generic/`.
- **`port.mk`.** In `armv7m/` alone: what its assembly needs from the assembler.

`generic/` holds only its `NOTICE` and the four functions in C, which work one byte at a time and so need nothing of the processor. Its `NOTICE` also lists the processors for which no assembly was found under a licence libterae can ship.

These files are outside the rules the C of the library follows: they are kept as their projects wrote them. Each licence they carry allows the library to ship under both of its licences.

source-structure-en.tex Page 17 November 8, 2026 13:46

4.3 How `PORT=auto` chooses

`src/port/select.in` is not C, and nothing includes it. With `PORT=auto`, the `Makefile` runs it through the compiler's preprocessor, with the flags of the build, and it leaves the name of one folder: the first whose test, on the macros the compiler defines, fits the processor. One that fits none gets `generic`, and so does any compiler that doesn't make ELF objects, as on macOS and Windows, since every folder of assembly is written for ELF.

With a folder of assembly, `base_string.c` copies with that folder's `memcpy` rather than its own loop, so the library then needs `libterae_port.a`.

5

The Platform

What `src/` leaves out because each system does it its own way is in `platform/`, the specialisation of the library on a system. It has the sockets and the waiting for them, the clock, random bytes, the program's output and stopping, TLS, and the stores that keep Session state across a restart, and with them the network loop of a Server and of a Client. It is the only code of the library that calls a system. A program can leave it out, with `OS=none`, and join `src/` to a loop of its own.

5.1 Two levels

Figure 5.1 shows the folder in two levels.

- **The top of `platform/`.** Written only against the headers that a system's folder carries out, so it is the same on every system: `base_platform.h`, `platform_wait.h`, and for a store `platform_store.h`, files, or `platform_flash.h`, flash. It includes no header of a system: only standard headers of C and, in `platform_tls.c`, those of Mbed TLS.
- **A folder for each system.** What that system does its own way, written against what the system documents and nothing else. Each one carries out `base_platform.h` for its system, and the headers of the top that its build needs.

source-structure-en.tex Page 20 November 8, 2026 13:46

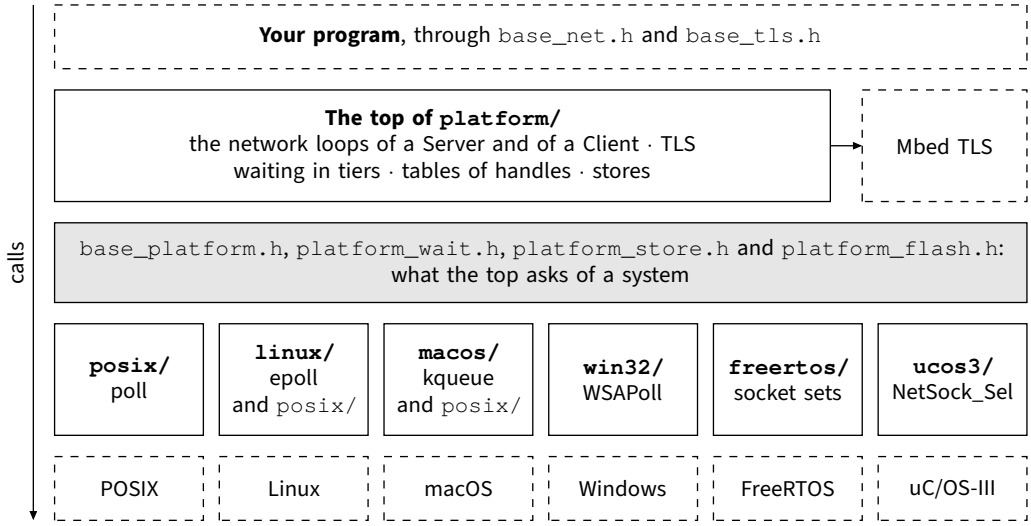


Figure 5.1 `platform/`, from the program down to the system. The top also calls the Server and Client Interfaces, the Tiers and WebSocket, as Figure 1.1 shows

5.2 The files at the top

Table 5.1 The files at the top of `platform/`

File	What it does	Built for
The network loops: `base_net.h`		
`platform_net_server.c`	Runs a Server on real connections, on one thread: waits for the sockets that are ready, reads them, and writes out what the Server has for each, over TCP, TLS or WebSocket.	Every system
`platform_net_client.c`	Runs a Client on one connection.	Every system
`platform_net_runs.c`	Writes runs of bytes, on a socket or through TLS, for both.	Every system
`platform_net_store.c`	The store of a Server or a Client that runs in those loops: restoring it, closing and syncing the records of each round, and writing the snapshots.	Every system
`platform_net.h`	What the loops and their store share.	–

continues

source-structure-en.tex Page 21 November 8, 2026 13:46

Table 5.1 The files at the top of `platform/` (*continued*)

File	What it does	Built for
TLS: `base_tls.h`		
`platform_tls.c`	TLS through Mbed TLS, with its memory from a static pool. Built without TLS, each function reports `MQTT_TLS_ERROR`.	Every system
`mbedtls_config.h`	What libterae adds to Mbed TLS's configuration. Mbed TLS is built with it.	Mbed TLS
Waiting in tiers: `base_platform.h`		
`platform_wait.c`	Waits in tiers on a system that takes an array of entries, one for each socket, as `poll` does.	`posix`, `win32`, `ucos3`
`platform_wait_set.c`	Waits on a system that keeps the set of sockets itself and reports those that are ready, as `epoll` and `kqueue` do.	`linux`, `macos`, `freertos`
`platform_wait.h`	What the two need from a system's folder.	–
Stores: `base_platform.h`		
`platform_store_file.c`	Keeps a store in a folder of files: the copies of the snapshot and two journal files, set aside into a folder of their own.	`posix`, `linux`, `macos`, `win32`
`platform_store.h`	The files it needs from a system's folder.	–
`platform_store_flash.c`	Keeps a store in a region of flash that the board names, cut into places of whole erase blocks.	`freertos`, `ucos3`
`platform_flash.h`	The flash it needs from a system's folder, which the board gives.	–
The rest of `base_platform.h`		
`platform_slots.c`, `platform_slots.h`	A table of handles, given out from a free list, for a system whose sockets aren't small numbers.	`win32`, `freertos`
`platform_parse.c`	Reads numbers, and the names of durabilities, from text, which needs nothing from a system.	Every system

continues

source-structure-en.tex Page 22 November 8, 2026 13:46

Table 5.1 The files at the top of `platform/` (*continued*)

File	What it does	Built for
`platform_probe_none.c`	The calls that `base_platform.h` keeps for tests and benchmarks, for a system with no processes to copy and no signals: each says it can't.	`win32`, `freertos`, `ucos3`

5.3 The folders of the systems

Each folder's files have the same roles, named by the same word after the folder's name: `_socket.c` for the sockets; `_system.c` for the clock, sleeping, the program's output and stopping, which `posix/` splits into `posix_clock.c` and `posix_signal.c`; `_random.c` for random bytes; one file for waiting, which gives `platform_wait.c` its entries or keeps the set of `platform_wait_set.c`; and for a store, `_file.c` and `_sync.c` for files, or `_flash.c` for flash. A folder's `_internal.h` is what its files share.

Table 5.2 The folders of the systems

Folder	What it uses	What a build takes
`posix`	POSIX.1-2008 alone, and `poll`, with two exceptions: random bytes from `/dev/urandom`, a file POSIX doesn't name, and `fsync`, from POSIX's optional File Synchronization.	Its ten files: `posix_socket.c`, `posix_options.c`, `posix_signal.c`, `posix_clock.c`, `posix_wait_one.c`, `posix_probe.c`, `posix_file.c`, `posix_poll.c`, `posix_random.c`, `posix_sync.c`; and `platform_wait.c`, `platform_store_file.c`
`linux`	`posix/`, with `epoll` for waiting and `getrandom` for random bytes.	`linux_epoll.c`, `linux_random.c`; the files of `posix/` but `posix_poll.c` and `posix_random.c`; and `platform_wait_set.c`, `platform_store_file.c`
`macos`	`posix/`, with `kqueue` for waiting, `getentropy` for random bytes, and `F_FULLFSYNC` to sync a file, as `fsync` there doesn't reach the drive's own cache.	`macos_kqueue.c`, `macos_random.c`, `macos_sync.c`; the files of `posix/` but `posix_poll.c`, `posix_random.c` and `posix_sync.c`; and `platform_wait_set.c`, `platform_store_file.c`

continues

source-structure-en.tex Page 23 November 8, 2026 13:46

Table 5.2 The folders of the systems (*continued*)

Folder	What it uses	What a build takes
`win32`	Winsock 2 and the Windows API, 32-bit and 64-bit, waiting with `WSAPoll`. Random bytes from `BCryptGenRandom`.	`win32_socket.c`, `win32_system.c`, `win32_poll.c`, `win32_random.c`, `win32_file.c`; and `platform_wait.c`, `platform_slots.c`, `platform_probe_none.c`, `platform_store_file.c`
`freertos`	The kernel and FreeRTOS+TCP, waiting with its socket sets. Random bytes from the program, through `xApplication GetRandomNumber`, and a store's flash from its board, through the `freertos_board_store_` calls.	`freertos_socket.c`, `freertos_system.c`, `freertos_select.c`, `freertos_random.c`, `freertos_flash.c`; and `platform_wait_set.c`, `platform_slots.c`, `platform_probe_none.c`, `platform_store_flash.c`
`ucos3`	uC/OS-III and uC/TCP-IP, waiting with `NetSock_Sel`. Random bytes from the program's board, through `ucos3_board_random`, and a store's flash through the `ucos3_board_store_` calls.	`ucos3_socket.c`, `ucos3_system.c`, `ucos3_select.c`, `ucos3_random.c`, `ucos3_flash.c`; and `platform_wait.c`, `platform_probe_none.c`, `platform_store_flash.c`

Every build of a system also takes the six files that Table 5.1 marks for every system.

5.4 The Makefile picks the files

No file of `platform/` asks which system it is on. `make OS=`*folder* takes the files of Table 5.2 for that folder, and compiles each with the definitions of its own folder: the top of `platform/` with none, `posix/`, `linux/` and `macos/` with `_POSIX_C_SOURCE`, `win32/` with the version of Windows it needs, and the real-time systems with the headers the program names in `RTOS_INCLUDES`. Without `OS`, the build takes the folder of the machine it runs on. `OS=none` takes nothing of `platform/`, and so no TLS. *Building libterae* describes each of these.

source-structure-en.tex Page 24 November 8, 2026 13:46

A new system is a new folder, which carries out `base_platform.h`, chooses `platform_wait.c` or `platform_wait_set.c` by how the system waits for sockets, and `platform_store_file.c` or `platform_store_flash.c` by where it keeps a store, carrying out `platform_store.h` or `platform_flash.h` for it, and is given its list of files in the `Makefile`.



6

Where to Look

Table 6.1 names the file to open first for each part of the library. The file's opening comment then names the others it works with.

Table 6.1 Where each part of the library is

For	Open
Using the library	
Making a Server, and the memory it is given	`include/base_server.h`
Making a Client	`include/base_client.h`
Letting `platform/` run the loop	`include/base_net.h`
Keeping Session state across a restart	`include/base_store.h`
The outcome of a call	`include/base_status.h`
The version, and whether a shared library fits the program	`include/base_library.h`

The Reason Codes and return codes	`include/v5_rc.h`
	`include/v311_rc.h`
	`include/v31_rc.h`

The Server	
The checks on a packet, and what it means	`src/base_server.c`
Cutting bytes into packets	`src/base_server_receive.c`
CONNECT, AUTH and Assigned Client Identifiers	`src/base_server_connect.c`

A message, from PUBLISH to its subscribers	`src/base_server_publish.c`
	`src/base_server_route.c`
	`src/base_tree.c`

continues

source-structure-en.tex Page 26 November 8, 2026 13:46

Table 6.1 Where each part of the library is (*continued*)

For	Open
Sessions and retained messages	`src/base_server_session.c`

The bytes in transit, and how many each connection may hold	`src/base_pool.c`
	`src/base_ring.c`
	`src/base_server_pool.c`

Its store: the journal, the snapshot, and a restore	`src/base_server_store.c`
	`src/base_server_snapshot.c`
	`src/base_server_restore.c`

The Client	
Connecting, publishing, and the messages held	`src/base_client.c`
What the Server sends	`src/base_client_receive.c`
Subscribing	`src/base_client_subscribe.c`
Its store	`src/base_client_store.c` `src/base_client_restore.c`
The bytes of the protocol	
The Control Packets	`src/base_packet.c`
Integers, strings and Binary Data	`src/base_wire.c`
UTF-8, Topic Names and Topic Filters	`src/base_utf8_check.c` `src/base_topic_check.c`
The Properties of MQTT 5.0	`src/v5_properties.c`
WebSocket	`src/base_websocket.c`
The records of a store, and their checks	`src/base_store.c`
A system	
The network loop of a Server	`platform/platform_net_server.c`

Waiting for sockets	`src/base_tier.c`
	`platform/platform_wait.c`
	`platform/platform_wait_set.c`

The sockets of one system	`platform/`*`os`*`/`, the file that ends in `_socket.c`
TLS	`platform/platform_tls.c` `platform/mbedtls_config.h`

A store in files or in flash	`platform/platform_net_store.c`
	`platform/platform_store_file.c`
	`platform/platform_store_flash.c`

continues

source-structure-en.tex Page 27 November 8, 2026 13:46

Table 6.1 Where each part of the library is (*continued*)

For	Open
The memory functions for a processor	`src/port/`*folder*`/NOTICE`